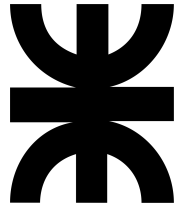


Universidad Tecnológica Nacional

Tecnicatura Universitaria en Programación



Trabajo Práctico Integrador

Programación I

Gestión de Datos de Países en Python:
filtros, ordenamientos y estadísticas

Integrantes: Ignacio **Tello**, Luca **Chialvo**.

Comisión 1

Grupo 482

Fecha de entrega: **17/06/2026**

Índice

Introducción.....	3
Marco teórico.....	3
Listas en Python.....	3
Diccionarios en Python.....	4
Funciones en Python.....	5
Condicionales en Python.....	7
Ordenamientos en Python.....	8
Estadísticas básicas en Python.....	9
Archivos CSV en Python.....	10
Decisiones Técnicas y Arquitectura.....	11
Organización del proyecto.....	11
Diagrama de flujo.....	12
Ejemplos de ejecución.....	14
Dificultades y Conclusiones.....	18
Bibliografía / Webgrafía.....	19
Enlaces relativos al proyecto.....	19
Referencias.....	19

Introducción

El presente trabajo tiene como finalidad exponer el proceso de desarrollo y los procedimientos llevados a cabo por el equipo de trabajo perteneciente al grupo número 482, integrado por Ignacio Tello y Luca Chialvo, estudiantes de la Tecnicatura Universitaria en Programación de la Universidad Tecnológica Nacional, para la elaboración del programa correspondiente al trabajo final de la asignatura Programación I.

El proyecto consiste en el desarrollo de un programa escrito en lenguaje Python, ejecutado mediante la terminal, cuyo objetivo es realizar diversas operaciones y funcionalidades sobre un archivo con formato .csv . Para su implementación se aplicaron los conocimientos, conceptos y fundamentos teóricos abordados durante el cursado de la materia.

El presente documento describe los fundamentos teóricos que sustentan las herramientas y técnicas utilizadas en el desarrollo del programa, además de detallar las decisiones tomadas durante el proceso de implementación, la organización del trabajo y las dificultades que surgieron a lo largo del proyecto.

Finalmente, se incluye una conclusión en la que se reflejan los aprendizajes obtenidos por los integrantes del equipo, considerando los desafíos enfrentados y la experiencia adquirida durante el desarrollo del trabajo práctico.

Marco teórico

Esta sección tiene el propósito de explicar conceptos teóricos vistos durante el cursado de la materia. La información fue recolectada mediante los apuntes brindados por la cátedra y consultando la documentación oficial de Python.

Listas en Python

Las listas constituyen una de las estructuras de datos fundamentales del lenguaje Python y representan una colección ordenada y mutable de elementos. El término *mutable* refiere a la capacidad de modificar el contenido de la estructura luego de su creación, permitiendo agregar, eliminar o reemplazar elementos sin necesidad de redefinir la variable. Las listas son especialmente relevantes en programación debido a su versatilidad para almacenar conjuntos de datos heterogéneos, es decir, elementos de distintos tipos de datos dentro de una misma estructura.

Desde un punto de vista computacional, una lista funciona como una secuencia indexada, lo que implica que cada elemento ocupa una posición específica accesible mediante índices numéricos. En Python, la indexación comienza en cero, de modo que el primer elemento de una lista se encuentra en la posición `0`, el segundo en la posición `1`, y así sucesivamente. También se admite indexación negativa, utilizada para acceder a elementos desde el final de la secuencia.

La sintaxis de una lista consiste en encerrar los elementos entre corchetes (`[]`) y separarlos mediante comas. Por ejemplo:

```
numeros = [10, 20, 30, 40]
```

En este caso, la variable `numeros` contiene una secuencia ordenada de valores enteros. El orden de almacenamiento posee importancia semántica, ya que determina la posición relativa de cada dato y afecta operaciones posteriores, como iteraciones u ordenamientos.

Las listas permiten ejecutar numerosas operaciones integradas. Entre las más relevantes se encuentran la inserción de nuevos elementos mediante métodos como `append()` o `insert()`, la eliminación de datos utilizando `remove()` o `pop()`, la concatenación de múltiples listas y la extracción parcial de segmentos a través de técnicas de *slicing*. Esta flexibilidad convierte a las listas en un recurso central para la manipulación dinámica de datos.

Desde una perspectiva algorítmica, las listas son utilizadas para representar vectores, secuencias temporales, conjuntos de registros y estructuras intermedias necesarias para el procesamiento de información. En aplicaciones de análisis de datos, por ejemplo, suelen emplearse para almacenar observaciones numéricas, mientras que en sistemas administrativos pueden contener nombres, identificadores o registros asociados a usuarios.

Diccionarios en Python

Los diccionarios representan una estructura de datos basada en asociaciones entre claves (*keys*) y valores (*values*). A diferencia de las listas, donde el acceso a los elementos depende de una posición numérica, en un diccionario cada dato es recuperado a partir de una clave única que funciona como identificador.

Esta característica convierte a los diccionarios en una herramienta particularmente eficiente para modelar información estructurada. Conceptualmente, pueden interpretarse como tablas de correspondencia o mapas de datos, donde cada atributo posee un valor asociado. Por ejemplo,

resulta adecuado representar la información de un estudiante mediante atributos como nombre, edad o carrera.

La sintaxis de un diccionario emplea llaves (`{}`), estableciendo relaciones clave-valor mediante el símbolo dos puntos (`:`):

```
estudiante = {  
    "nombre": "Ana",  
    "edad": 22,  
    "carrera": "Ingeniería"  
}
```

En este ejemplo, `"nombre"`, `"edad"` y `"carrera"` constituyen claves, mientras que sus respectivos contenidos representan valores asociados. El acceso a la información se realiza indicando la clave correspondiente:

```
print(estudiante["nombre"])
```

Un aspecto importante de los diccionarios es que las claves deben ser únicas e inmutables. Esto significa que no puede existir duplicación de claves dentro de una misma estructura y que deben pertenecer a tipos de datos no modificables, como cadenas de texto, enteros o tuplas.

Desde el punto de vista práctico, los diccionarios son ampliamente utilizados en aplicaciones de software donde se requiere representar entidades complejas. En programación orientada a datos permiten organizar configuraciones, bases de información, registros administrativos y respuestas provenientes de servicios web. Asimismo, su implementación interna optimiza la velocidad de búsqueda, lo cual favorece operaciones eficientes incluso sobre grandes volúmenes de información.

Funciones en Python

Las funciones constituyen bloques de código reutilizables diseñados para ejecutar tareas específicas dentro de un programa. Su principal propósito consiste en modularizar el desarrollo del software, favoreciendo la reutilización, legibilidad y mantenibilidad del código.

En términos conceptuales, una función puede interpretarse como una unidad lógica que recibe datos de entrada, realiza un procesamiento determinado y eventualmente devuelve un

resultado. Esta estructura permite dividir programas complejos en componentes más pequeños y especializados, reduciendo la redundancia y facilitando el control del flujo de ejecución.

En Python, las funciones se definen mediante la palabra reservada `def`, seguida por el nombre de la función y un conjunto opcional de parámetros:

```
def calcular_area(base, altura):  
    return base * altura
```

En este caso, la función `calcular_area()` recibe dos parámetros de entrada y retorna un valor calculado. La instrucción `return` cumple el rol de devolver el resultado al lugar desde el cual la función fue invocada.

Las funciones pueden clasificarse según diversos criterios. Existen funciones integradas (*built-in functions*), proporcionadas por el propio lenguaje, como `len()`, `sum()` o `print()`, y funciones definidas por el usuario, creadas para resolver necesidades específicas. También pueden diferenciar entre funciones con retorno y procedimientos sin valor devuelto explícitamente.

La importancia de las funciones trasciende la mera organización del código. En ingeniería de software representan un principio esencial de abstracción, entendido como la capacidad de encapsular complejidad interna y exponer únicamente una interfaz de uso. Gracias a esta propiedad, un programador puede reutilizar procedimientos complejos sin necesidad de comprender cada detalle de implementación en todo momento.

Adicionalmente, las funciones favorecen la escalabilidad del software, ya que permiten corregir errores o introducir mejoras modificando un único fragmento de código, sin afectar el resto del sistema.

Condicionales en Python

Las estructuras condicionales constituyen mecanismos de control de flujo utilizados para ejecutar instrucciones de forma selectiva según el cumplimiento de determinadas condiciones lógicas. En programación, un condicional representa un punto de decisión donde el sistema evalúa expresiones booleanas —es decir, expresiones cuyo resultado puede ser verdadero (`True`) o falso (`False`)— para determinar el camino de ejecución.

En Python, los condicionales se implementan principalmente mediante las sentencias `if`, `elif` y `else`. La estructura `if` establece una condición principal, `elif` permite incorporar evaluaciones alternativas y `else` define un comportamiento por defecto cuando ninguna condición previa se cumple.

Por ejemplo:

```
edad = 20
if edad >= 18:
    print("Mayor de edad")
else:
    print("Menor de edad")
```

En este ejemplo, el programa evalúa la variable `edad` y ejecuta distintas instrucciones según el resultado lógico de la comparación.

Las condiciones pueden construirse utilizando operadores relacionales, tales como igualdad (`==`), desigualdad (`!=`), mayor que (`>`), menor que (`<`) o comparaciones inclusivas (`>=`, `<=`). Asimismo, es posible combinar múltiples expresiones mediante operadores lógicos como `and`, `or` y `not`.

Desde un punto de vista computacional, los condicionales resultan esenciales para desarrollar sistemas adaptativos capaces de responder a distintos escenarios de entrada. Su aplicación se observa en validaciones de usuarios, sistemas de recomendación, filtros de datos, automatización de procesos y toma de decisiones algorítmicas.

Ordenamientos en Python

El ordenamiento de datos corresponde al proceso de reorganizar elementos de acuerdo con un criterio específico, generalmente ascendente o descendente. Esta operación posee gran relevancia en informática debido a que facilita la búsqueda, comparación, interpretación y análisis de información.

Python incorpora mecanismos integrados para ordenar secuencias de datos, principalmente a través de la función `sorted()` y el método `sort()`. La diferencia principal entre ambos radica en que `sorted()` genera una nueva colección ordenada sin alterar la estructura original, mientras que `sort()` modifica directamente la lista existente.

Por ejemplo:

```
numeros = [8, 3, 1, 5]
numeros.sort()
```

Tras la ejecución, la lista quedará organizada de menor a mayor.

Uno de los aspectos más relevantes del ordenamiento en Python es la posibilidad de personalizar criterios mediante el parámetro *key*, permitiendo ordenar elementos complejos según atributos específicos. Esto resulta especialmente útil en listas de diccionarios o registros estructurados.

Desde una perspectiva algorítmica, el estudio de los ordenamientos involucra conceptos de eficiencia computacional. Diversos algoritmos clásicos, como *Bubble Sort*, *Merge Sort* o *Quick Sort*, presentan diferentes costos temporales y espaciales. Aunque Python abstrae esta complejidad mediante funciones integradas optimizadas, comprender el fundamento teórico de los ordenamientos resulta importante para seleccionar estrategias eficientes en conjuntos de datos extensos.

En términos prácticos, los ordenamientos permiten organizar registros académicos por calificación, clasificar productos por precio o estructurar bases de datos cronológicamente.

Estadísticas básicas en Python

Las estadísticas básicas comprenden un conjunto de procedimientos matemáticos orientados a describir, resumir e interpretar conjuntos de datos. En el contexto de Python, estas operaciones pueden realizarse mediante funciones integradas o bibliotecas especializadas orientadas al análisis cuantitativo.

Entre las medidas estadísticas más utilizadas se encuentran la media, mediana, moda, rango y desviación estándar. La media aritmética representa el promedio de un conjunto de valores; la mediana identifica el valor central de una distribución ordenada; la moda corresponde al dato de mayor frecuencia; mientras que la desviación estándar mide la dispersión respecto del promedio.

Python incorpora herramientas para realizar estos cálculos de forma eficiente. Una de las bibliotecas más utilizadas es *statistics*, integrada al lenguaje estándar, la cual proporciona funciones específicas para análisis descriptivo.

Por ejemplo:

```
import statistics
datos = [5, 7, 8, 10, 12]
promedio = statistics.mean(datos)
```

En este caso, la función `mean()` calcula automáticamente la media de los datos almacenados.

El análisis estadístico mediante programación resulta especialmente relevante en disciplinas científicas, económicas, sociales e ingenieriles, donde se requiere interpretar grandes cantidades de información empírica. La automatización de cálculos estadísticos reduce errores humanos y mejora significativamente la reproducibilidad de resultados.

Además de las medidas descriptivas, Python permite desarrollar análisis más avanzados mediante bibliotecas como `NumPy` y `Pandas`, utilizadas extensamente en ciencia de datos, inteligencia artificial y modelado predictivo.

Archivos CSV en Python

Los archivos CSV (*Comma-Separated Values*) representan un formato de almacenamiento de datos tabulares donde la información se organiza en filas y columnas separadas mediante delimitadores, generalmente comas. Debido a su simplicidad y compatibilidad, constituyen uno de los formatos más utilizados para el intercambio de datos entre aplicaciones, bases de datos y sistemas estadísticos.

Cada línea de un archivo CSV corresponde a un registro, mientras que los valores separados representan atributos específicos de dicho registro. Por ejemplo, un archivo puede almacenar información académica donde cada fila representa un estudiante y cada columna corresponde a nombre, edad o calificación.

Python proporciona herramientas específicas para leer y escribir archivos CSV mediante el módulo `csv`, incluido en la biblioteca estándar del lenguaje. La lectura de un archivo requiere abrirlo previamente y recorrer sus registros:

```
import csv
with open("datos.csv", "r") as archivo:
    lector = csv.reader(archivo)
```

La estructura `with` resulta relevante porque garantiza el cierre automático del archivo al finalizar la operación, optimizando el manejo de recursos del sistema.

El procesamiento de archivos CSV posee gran relevancia en contextos de análisis de datos debido a que muchas fuentes de información —incluyendo planillas de cálculo, exportaciones de bases de datos y sistemas empresariales— emplean este formato para almacenar registros.

En aplicaciones reales, los archivos CSV son frecuentemente utilizados para importar conjuntos de datos masivos, automatizar reportes, consolidar información estadística y alimentar sistemas de aprendizaje automático. Su simplicidad estructural y amplia interoperabilidad convierten a este formato en un estándar práctico para el manejo de datos en entornos computacionales.

Decisiones Técnicas y Arquitectura

Organización del proyecto

En cuanto a la estructura y arquitectura del programa, se adoptó una organización modular basada en separación de responsabilidades. El archivo `main.py` actúa como punto de entrada, encargándose exclusivamente de dos tareas: mostrar el menú de opciones al usuario y ejecutar el bucle principal que captura la selección y deriva cada acción a la función correspondiente del módulo de operaciones. Este archivo no contiene lógica de negocio, sino únicamente la coordinación del flujo del programa.

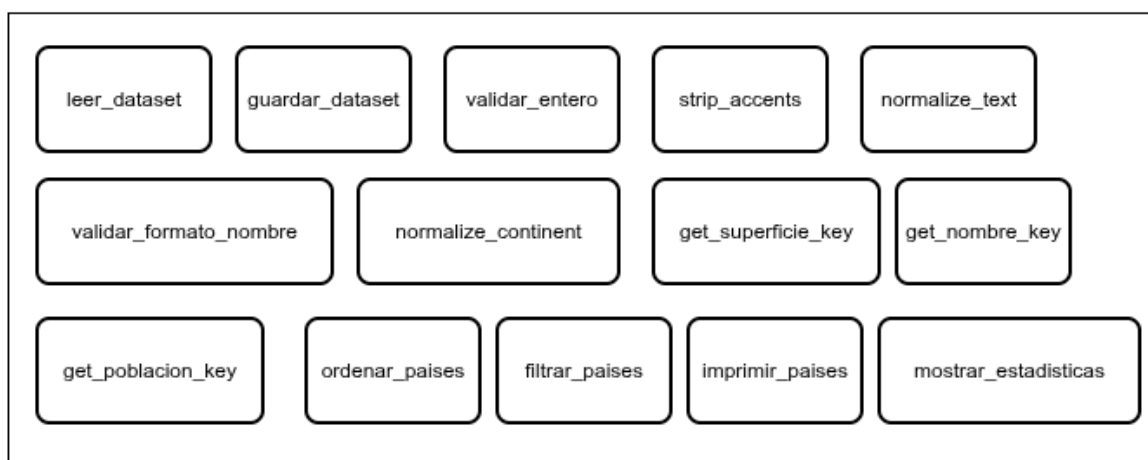
El módulo `operaciones.py` concentra toda la lógica relacionada con las operaciones del menú: agregar un nuevo país al dataset, actualizar sus datos (población y superficie), buscar países por nombre (con soporte para coincidencias parciales), y filtrar países según distintos criterios (continente, rango de población y rango de superficie). Cada función implementa la lógica completa de su operación, incluyendo validación de entradas, interacción con el usuario mediante `input/print`, y la persistencia de datos a través de las utilidades del módulo auxiliar.

Por último, `utils.py` funciona como un módulo de utilidades reutilizables que contiene funciones de propósito general: lectura y escritura del archivo CSV del dataset, normalización de texto (eliminación de acentos, conversión a Title Case), validación de formatos, ordenamiento por distintos criterios (nombre, población, superficie), filtrado flexible, impresión formateada de

resultados y cálculo de estadísticas básicas. Este diseño permite que operaciones.py se enfoque en la lógica específica de cada operación mientras que las operaciones de bajo nivel y comunes permanecen centralizadas y reutilizables en [utils.py](#).

En cuanto al dataset, este contiene 50 países con sus nombres, población, superficie y continente. Decidimos usar un nombre exclusivo para cada continente en lugar de utilizar sus posibles variables y respetar las tildes y mayúsculas de los nombres. Esto es, que utilizamos el continente América en lugar de separarla en América del sur, Centroamérica y América del norte.

El archivo [utils.py](#) posee las funciones:



El archivo [operaciones.py](#) posee las funciones:

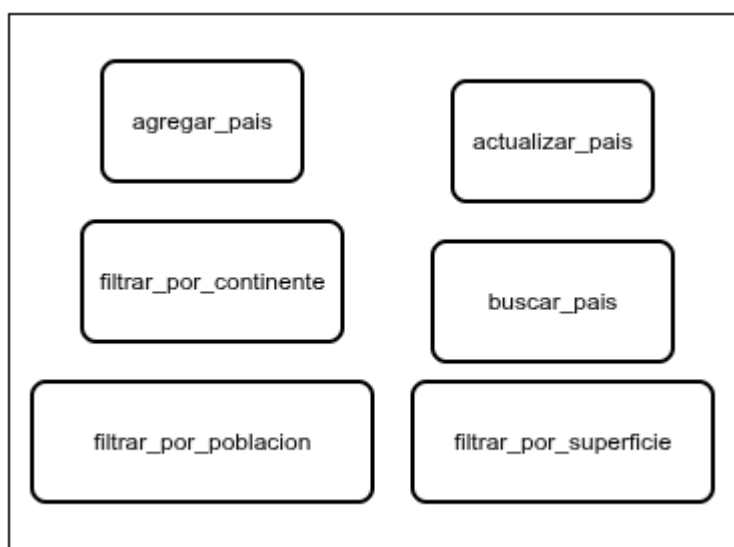
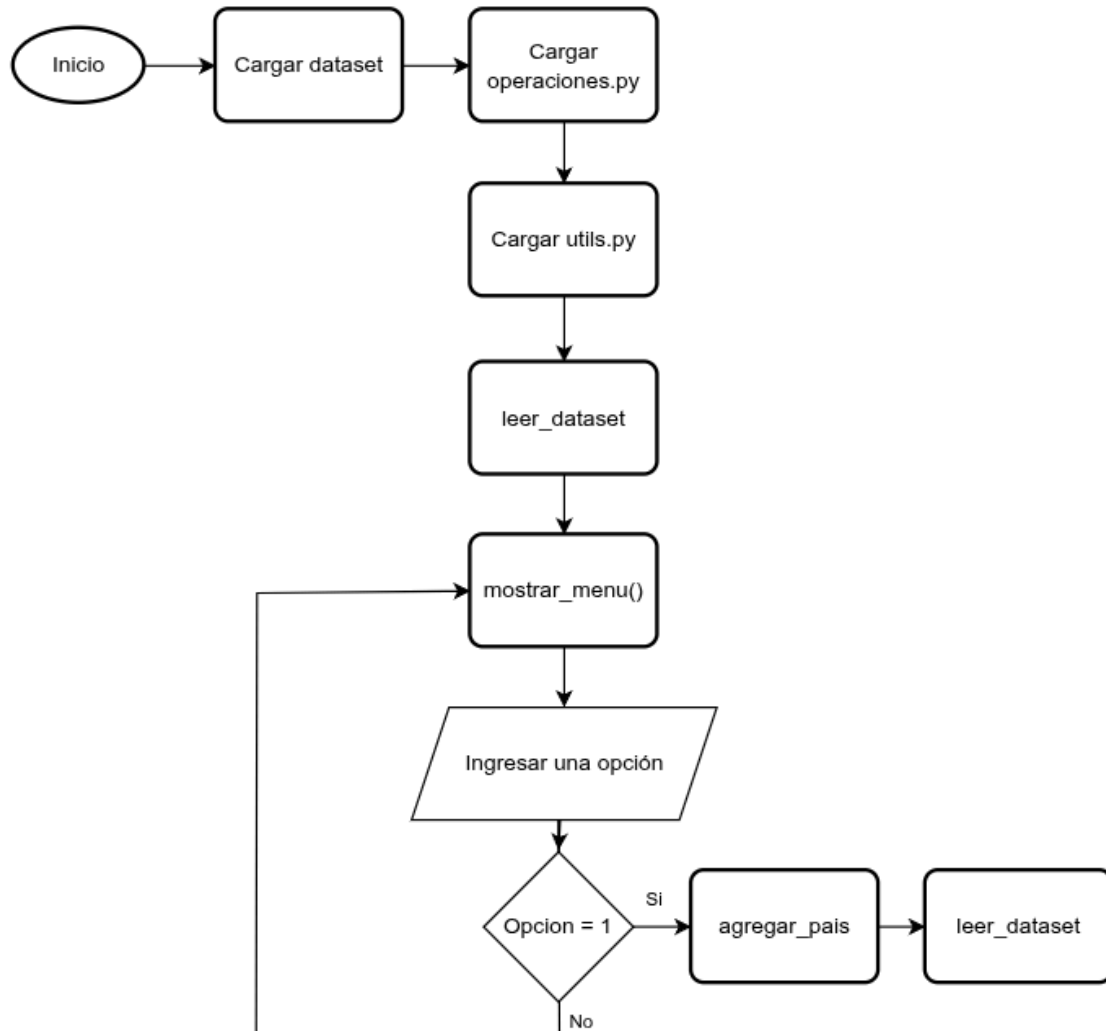
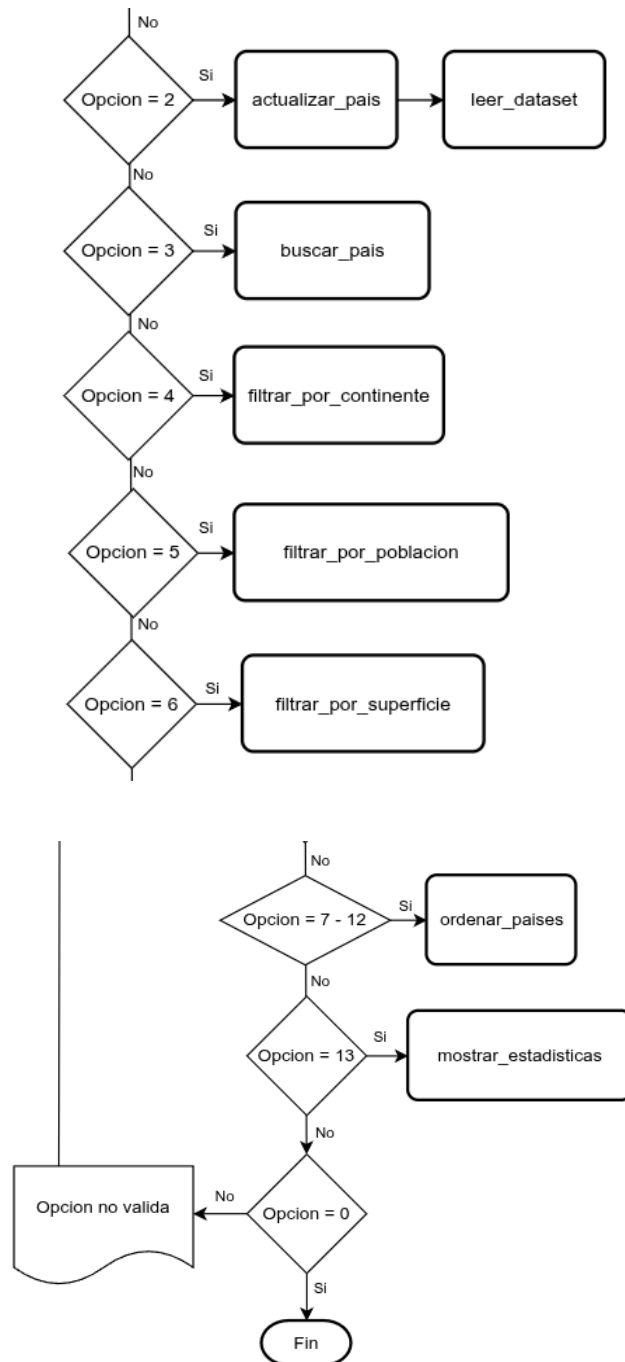


Diagrama de flujo

Podemos representar el flujo de ejecución del programa de la siguiente manera:





Ejemplos de ejecución

Lo primero que podemos ver al ejecutar el programa es un menú interactivo que se muestra hasta que terminemos la ejecución (ya sea por medio de la opción 0, o una interrupción por teclado). Cada opción se elige ingresando por teclado un número del 0 al 13, correspondiente a una funcionalidad del programa.

```
Menú de opciones:
1. Agregar país
2. Actualizar país
3. Buscar país por nombre
4. Filtrar por continente
5. Filtrar por rango de población
6. Filtrar por rango de superficie
7. Ordenar por nombre ascendente
8. Ordenar por nombre descendente
9. Ordenar por población ascendente
10. Ordenar por población descendente
11. Ordenar por superficie ascendente
12. Ordenar por superficie descendente
13. Mostrar estadísticas
0. Salir
Seleccione una opción: █
```

La opción número uno del menú nos permite agregar un país dentro de nuestra base de datos en formato CSV (*Comma Separated Values*), esto mediante *inputs* de usuario utilizando la consola.

```
Menú de opciones:
1. Agregar país
2. Actualizar país
3. Buscar país por nombre
4. Filtrar por continente
5. Filtrar por rango de población
6. Filtrar por rango de superficie
7. Ordenar por nombre ascendente
8. Ordenar por nombre descendente
9. Ordenar por población ascendente
10. Ordenar por población descendente
11. Ordenar por superficie ascendente
12. Ordenar por superficie descendente
13. Mostrar estadísticas
0. Salir
Seleccione una opción: 1
Nombre: cabo verde
Población: 530000
Superficie: 4000
Continente: africa
País 'Cabo Verde' agregado correctamente.
```

Resultado de la ejecución dentro del archivo database.csv:

```
49 Nueva Zelanda,5300000,268838,Oceanía
50 Papúa Nueva Guinea,10000000,462840,Oceanía
51 Fiyi,930000,18274,Oceanía
52 Cabo Verde,530000,4000,África
53
```

Como podemos observar, los datos siempre se agregan al final de la base de datos respetando el formato esperado.

Si ingresamos un país que ya existe en la base de datos:

```

Seleccione una opción: 1
Nombre: argentina
El país 'Argentina' ya existe en el dataset. Ingrese otro nombre.
Nombre:
    
```

Otra prueba que podemos hacer es la opción *Buscar país por nombre*, que nos permite buscar dentro de nuestra base de datos coincidencias exactas o parciales y mostrar los datos que le corresponden al país que buscamos. Esto significa que podemos buscar *Argentina* como “argentina”, “arg”, “argen” o “ntina” y encontrará el país de igual forma.

Ejemplos de ejecución:

```

Seleccione una opción: 3
Ingrese el nombre a buscar: ntina
Nombre                Población    Superficie    Continente
-----
Argentina              46000000    2780400       América
    
```

```

Seleccione una opción: 3
Ingrese el nombre a buscar: argen
Nombre                Población    Superficie    Continente
-----
Argentina              46000000    2780400       América
    
```

También podemos observar que se mostrarán todos los países que coincidan con la búsqueda del usuario. Es decir, buscar “arg” mostrará los datos tanto de *Argentina* como *Argelia* ya que coinciden en su prefijo.

```

Seleccione una opción: 3
Ingrese el nombre a buscar: arg
Nombre                Población    Superficie    Continente
-----
Argentina              46000000    2780400       América
Argelia                45000000    2381741       África
    
```

Ahora, si elegimos la opción 4, podremos filtrar los países según su continente. Dicho continente lo debemos escribir por teclado y se busca una coincidencia parcial o total.

Seleccione una opción: 4
 Continente: america
 Países en América:

Nombre	Población	Superficie	Continente
Argentina	46000000	2780400	América
Brasil	203000000	8515767	América
Chile	19600000	756102	América
Uruguay	3500000	176215	América
Paraguay	6800000	406752	América
Bolivia	12300000	1098581	América
Perú	34000000	1285216	América
Colombia	52000000	1141748	América
Venezuela	28000000	916445	América
Ecuador	18000000	283561	América
México	129000000	1964375	América
Estados Unidos	340000000	9833517	América
Canadá	41000000	9984670	América
Cuba	11000000	109884	América
Guatemala	18000000	108889	América

También eligiendo la opción número 7 podremos listar todos los países en orden alfabético ascendente (A-Z):

Seleccione una opción: 7

Nombre	Población	Superficie	Continente
Alemania	84000000	357588	Europa
Arabia Saudita	36000000	2149690	Asia
Argelia	45000000	2381741	África
Argentina	46000000	2780400	América
Australia	27000000	7692024	Oceanía
Bolivia	12300000	1098581	América
Brasil	203000000	8515767	América
Bélgica	11800000	30528	Europa
Cabo Verde	530000	4000	África
Canadá	41000000	9984670	América
Chile	19600000	756102	América
China	1410000000	9596961	Asia
Colombia	52000000	1141748	América
Corea del Sur	52000000	100210	Asia
Cuba	11000000	109884	América
Ecuador	18000000	283561	América

Otra opción interesante es la número 13, que muestra estadísticas relevantes sobre los países, como el país con más y menos población, o países mas o menos extensos:

Seleccione una opción: 13
 Estadísticas del dataset de países:

- País con mayor población: India (1430000000)
- País con menor población: Cabo Verde (530000)
- Promedio de población: 114954117.65
- Promedio de superficie: 1863978.18

Finalmente, demostramos que al elegir la opción 0, que permite terminar con la ejecución del programa:

```
jo Integrador Programacion 1/main.py"
Menú de opciones:
 1. Agregar país
 2. Actualizar país
 3. Buscar país por nombre
 4. Filtrar por continente
 5. Filtrar por rango de población
 6. Filtrar por rango de superficie
 7. Ordenar por nombre ascendente
 8. Ordenar por nombre descendente
 9. Ordenar por población ascendente
10. Ordenar por población descendente
11. Ordenar por superficie ascendente
12. Ordenar por superficie descendente
13. Mostrar estadísticas
 0. Salir
Seleccione una opción: 0
Saliendo...
PS D:\Trabajo Integrador Programacion 1> █
```

Dificultades y Conclusiones

Durante el desarrollo del proyecto se presentaron diversas dificultades relacionadas, principalmente, con la organización inicial del trabajo. En una primera etapa fue necesario establecer acuerdos sobre la planificación, la distribución de tareas y las responsabilidades que asumiría cada integrante del equipo, con el objetivo de alcanzar un resultado final coherente y organizado.

Una vez creado el repositorio en GitHub, se realizaron pruebas iniciales mediante modificaciones simples en los archivos, con el propósito de verificar el correcto funcionamiento del sistema de control de versiones y asegurar que los integrantes pudieran realizar aportes de manera adecuada, evitando inconvenientes durante el desarrollo colaborativo.

A medida que el proyecto fue incorporando nuevas funcionalidades, surgió la necesidad de mejorar la estructura del código. Se observó que el archivo principal comenzaba a volverse demasiado extenso y complejo de mantener, por lo que se decidió implementar una organización modular mediante la separación de funcionalidades en los archivos `operaciones.py` y `utils.py`. Esta decisión permitió mejorar la legibilidad del código, facilitar el mantenimiento y optimizar el trabajo conjunto entre los integrantes del equipo.

Asimismo, se identificó que el trabajo colaborativo requiere una comunicación constante y una adecuada toma de decisiones. La coordinación entre los miembros resultó fundamental para

resolver dificultades, definir criterios comunes y avanzar de manera ordenada en el desarrollo del proyecto.

A pesar de los desafíos encontrados, la finalización del trabajo generó una experiencia satisfactoria al poder observar el funcionamiento del resultado obtenido. La ejecución del código representa no sólo la implementación de los contenidos abordados durante la materia, sino también la integración de los conocimientos adquiridos y la comprensión de los conceptos teóricos aplicados en cada una de las etapas del desarrollo.

Bibliografía / Webgrafía

Enlaces relativos al proyecto

Enlace al repositorio de GitHub: <https://github.com/lucachialvo/Trabajo-integrador---Programacion-1>

Enlace al video demostrativo: https://youtu.be/Q4xKmF_guaQ

Referencias

Python Software Foundation. (2026). *Python Documentation*. <https://docs.python.org/3/>

Universidad Tecnológica Nacional. (2025). *Archivos*. Cátedra Programación 1, Tecnicatura Universitaria en Programación.

https://drive.google.com/file/d/1PnAbJC65b5tkWhqn6oEZq4Q_aL_mdJLI/view?pli=1

Universidad Tecnológica Nacional. (2025). *Condicionales*. Cátedra Programación 1, Tecnicatura Universitaria en Programación.

https://tup.sied.utn.edu.ar/pluginfile.php/25705/mod_label/intro/Condicionales.pdf

Universidad Tecnológica Nacional. (2025). *Diccionarios*. Cátedra Programación 1, Tecnicatura Universitaria en Programación.

https://tup.sied.utn.edu.ar/pluginfile.php/25804/mod_label/intro/Diccionarios%20-%20TUPaD.docx%20%281%29.pdf

Universidad Tecnológica Nacional. (2025). *Funciones*. Cátedra Programación 1, Tecnicatura Universitaria en Programación.

https://tup.sied.utn.edu.ar/pluginfile.php/25771/mod_label/intro/5%20-%20Funciones.docx
.pdf

Universidad Tecnológica Nacional. (2025). *Listas - Actividad 2*. Cátedra Programación 1, Tecnicatura

Universitaria en Programación.

https://tup.sied.utn.edu.ar/pluginfile.php/25748/mod_label/intro/Actividad%20-%20Listas.pdf

Universidad Tecnológica Nacional. (2025). *Listas - Actividad 4*. Cátedra Programación 1, Tecnicatura

Universitaria en Programación.

https://tup.sied.utn.edu.ar/pluginfile.php/25753/mod_label/intro/Listas%20Actividad%204.pdf